claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-a68a441df32cc3eb8.jsonl`
- SHA-256: `d0291506a9041ca073b6f5025512f2ecd5e01c054018d91109311441463d803d`
- Source modified: `2026-06-18T01:36:34+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:33:12.222Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

CLASS: SQLite migration ladder & transactions (backend/infrastructure/database.py + callers). PRAGMA user_version steps, idempotency/ordering, partial-migration on error, missing commit/rollback, schema/row contract drift vs read sites.

2. assistant (2026-06-18T01:33:14.889Z)

I'll hunt for bugs in the SQLite migration ladder and transaction handling. Let me start by reading the database infrastructure.

3. user (2026-06-18T01:33:15.642Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
```

```
                with conn:
                    yield conn
            finally:
                conn.close()

        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                # Videos Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS videos (
                        id TEXT PRIMARY KEY,
                        filepath TEXT NOT NULL,
                        processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                        status TEXT NOT NULL,
                        validation_results TEXT
                    )
                """)

                # Play Segments Table (Extensible metadata JSON)
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS play_segments (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        video_id TEXT NOT NULL,
                        start_time REAL NOT NULL,
                        end_time REAL NOT NULL,
                        duration REAL NOT NULL,
                        server TEXT,
                        receiver TEXT,
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)
```

```
87
88                  # Versioned migrations live here. PRAGMA user_version is the schema
89                  # contract ? bump it for every change and add an ordered `if version < N`
90                  # block below. Tests assert the expected version, so accidental drift fails
CI.
91                  version = cursor.execute("PRAGMA user_version").fetchone()[0]
92
93                  if version < 1:
94                      # v1: raw candidate detections (pre-confirmation), detector-agnostic.
95                      # Common fields are columns; detector-specific metrics go in `stats`
JSON,
96                      # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
97                      cursor.execute("""
98                          CREATE TABLE IF NOT EXISTS candidate_segments (
99                              id INTEGER PRIMARY KEY AUTOINCREMENT,
100                             video_id TEXT NOT NULL,
101                             detector TEXT NOT NULL,
102                             chunk_index INTEGER,
103                             start_time REAL NOT NULL,
104                             end_time REAL NOT NULL,
105                             duration REAL NOT NULL,
106                             confidence REAL,
107                             stats TEXT,
108                             detection_params TEXT,
109                             confirmed_segment_id INTEGER,
110                             human_label TEXT,
111                             notes TEXT,
112                             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113                             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114                             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115                         )
116                     """)
117                     cursor.execute("""
118                         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119                             ON candidate_segments(video_id, detector)
120                     """)
121                     cursor.execute("PRAGMA user_version = 1")
122
123                 if version < 2:
124                     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125                     # /api/process request. `status` is the EXECUTION state of the job
126                     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127                     # that failed validation) lives inside `result` JSON as
result["status"].
128                     # `result` is the full sync-era response dict (incl. report paths), so
the
129                     # observability report is the job's artifact, per the plan.
130                     cursor.execute("""
131                         CREATE TABLE IF NOT EXISTS jobs (
132                             id TEXT PRIMARY KEY,
133                             video_path TEXT NOT NULL,
134                             video_id TEXT,
135                             segmenter TEXT,
136                             player_pool TEXT,
137                             max_frames INTEGER,
138                             status TEXT NOT NULL DEFAULT 'queued',
139                             progress TEXT,
140                             error TEXT,
141                             result TEXT,
142                             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                             started_at TIMESTAMP,
144                             finished_at TIMESTAMP
```

```
                        )
                    """)
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
                    cursor.execute("PRAGMA user_version = 2")

                if version < 3:
                    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
                    # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
                    # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
                    # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
                    # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
                    cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
                    cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                    cursor.execute("PRAGMA user_version = 3")

                if version < 4:
                    # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
                    # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
                    # path can scope rows to an owner. **NULL = local install = byte-
identical to
                    # today** ? every existing row stays NULL and every existing query
(which never
                    # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
                    # Additive ONLY: no served-behavior change rides on this slice.
                    cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
                    cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
                    # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
                    # of index churn while making the eventual per-user lookups cheap.
                    cursor.execute(
                        "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
                        "WHERE user_id IS NOT NULL")
                    cursor.execute(
                        "CREATE INDEX IF NOT EXISTS idx_candidates_user "
                        "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
                    cursor.execute("PRAGMA user_version = 4")

                if version < 5:
                    # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
                    # version): the resolved per-user `fusion_config` overlay + an INLINE
                    # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
                    # and provenance that earned it. `is_active` pins the one row the
serving bridge
                    # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
                    # only persists/loads it ? NOTHING reads it on the served path yet.
                    #
                    #   user_id              ? owner of the model (NULL allowed = the local
install)
                    #   version              ? monotonic per-user model version
                    #   baseline_version     ? the shared baseline this was fit against
(upgrade switch)
                    #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-
```

```
MAJOR re-fit gate
192                     #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193                     #   classifier_json      ? JSON LogisticRegression.to_dict() (None =
config-only model)
194                     #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it
was promoted)
195                     #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n
trust floor input)
196                     #   is_active            ? the one resolved row for this user (partial-
unique below)
197                     cursor.execute("""
198                         CREATE TABLE IF NOT EXISTS user_models (
199                             id INTEGER PRIMARY KEY AUTOINCREMENT,
200                             user_id TEXT,
201                             version INTEGER NOT NULL DEFAULT 1,
202                             baseline_version TEXT,
203                             feature_schema_hash TEXT,
204                             fusion_config TEXT,
205                             classifier_json TEXT,
206                             promotion_evidence TEXT,
207                             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
208                             is_active INTEGER NOT NULL DEFAULT 0,
209                             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
210                         )
211                     """)
212                     # One model version per user (NULLs compare distinct in SQLite, so the
local
213                     # install can still carry multiple versioned rows; the active-row guard
below
214                     # is what enforces a single served model).
215                     cursor.execute("""
216                         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
217                             ON user_models(user_id, version)
218                     """)
219                     # At most ONE active model per user: a partial-unique index over the
active rows.
220                     # (SQLite treats NULL user_id rows as distinct, so the local install's
single
221                     # active row is still guarded ? there is one local user.)
222                     cursor.execute("""
223                         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
224                             ON user_models(user_id) WHERE is_active = 1
225                     """)
226                     cursor.execute("PRAGMA user_version = 5")
227                 # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
228
229                 conn.commit()
230
231         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
232             """Register or update a video row WITHOUT touching its child segments.
233
234             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
235             with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
236             That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
237             before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
238             row in place; clearing segments is now an explicit, deliberate operation
239             (clear_video_data / prune_segments).
240             """
241             try:
```

```
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
                        "ON CONFLICT(id) DO UPDATE SET "
                        "filepath=excluded.filepath, status=excluded.status, "
                        "validation_results=excluded.validation_results",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
```

```
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
```

```
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
```

```
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
                        "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
                        (job_id, video_path, segmenter,
                         json.dumps(player_pool) if player_pool is not None else None,
                         max_frames, json.dumps(policy) if policy is not None else None)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to create job {job_id}: {e}")
                return False

        def mark_job_running(self, job_id: str) -> bool:
```

```
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                        "progress='starting' WHERE id = ?", (job_id,))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} running: {e}")
                return False

        def set_job_progress(self, job_id: str, progress: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set progress for job {job_id}: {e}")
                return False

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set video_id for job {job_id}: {e}")
                return False

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='done', progress='finished', result=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                        (json.dumps(result), job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} done: {e}")
                return False

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='failed', error=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} failed: {e}")
                return False

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
```

```
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
        def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
            """Attach/replace a job's JSON ExecutionPolicy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET policy=? WHERE id = ?",
                        (json.dumps(policy) if policy is not None else None, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set policy for job {job_id}: {e}")
                return False

        def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
            """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
            worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
            computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='waiting', "
                        "next_poll_at=datetime('now', ?), "
                        "progress=COALESCE(?, progress) WHERE id = ?",
                        (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set job {job_id} waiting: {e}")
                return False

        def seconds_until_next_due(self) -> Optional[float]:
            """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
            schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
            already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
            try:
                with self._connect() as conn:
                    row = conn.cursor().execute(
                        "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
                        "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
                        "FROM jobs WHERE status='waiting'").fetchone()
                if row is None or row["secs"] is None:
                    return None
                return max(0.0, float(row["secs"]))
            except Exception as e:
```

```
588                 logger.error(f"Failed to compute next-due delay: {e}")
589                 return None
590
591         def claim_due_job(self) -> Optional[Dict[str, Any]]:
592             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
593             flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
594             double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
595             first. Returns the claimed job dict, or None if nothing is runnable."""
596             try:
597                 with self._connect() as conn:
598                     cur = conn.cursor()
599                     row = cur.execute(
600                         "SELECT id, status FROM jobs WHERE status='queued' "
601                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
602                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
603                     if not row:
604                         return None
605                     cur.execute(
606                         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
607                         "WHERE id=? AND status=?", (row["id"], row["status"]))
608                     conn.commit()
609                     if cur.rowcount != 1:
610                         return None                     # lost the race to a concurrent
worker
611                 return self.get_job(row["id"])
612             except Exception as e:
613                 logger.error(f"Failed to claim a due job: {e}")
614                 return None
615
616         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
617             """Crash recovery: any job still queued/running from a previous process is dead
618             (the executor is in-process). Mark failed so clients see a terminal state, not a
619             hang. Called once at startup. Returns the number of jobs swept."""
620             try:
621                 with self._connect() as conn:
622                     cur = conn.cursor()
623                     cur.execute(
624                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
625                         "WHERE status IN ('queued', 'running')", (reason,))
626                     conn.commit()
627                     return cur.rowcount
628             except Exception as e:
629                 logger.error(f"Failed to sweep stale jobs: {e}")
630                 return 0
631
632         # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
633         # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
634         # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
635         # serving.py) resolves them, and wiring that bridge into the production decision
seam
636         # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
637         # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
638         # JSON-serialised dicts; the rest are scalar columns.
639
640         def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
641                               baseline_version: Optional[str] = None,
642                               feature_schema_hash: Optional[str] = None,
643                               fusion_config: Optional[Dict[str, Any]] = None,
644                               classifier_json: Optional[Dict[str, Any]] = None,
```

```
                               promotion_evidence: Optional[Dict[str, Any]] = None,
                               n_videos_at_fit: int = 0,
                               is_active: bool = False) -> Optional[int]:
            """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.

            Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
            rowid). When ``is_active`` is True this row becomes the user's single active
model ?
            any previously-active row for the SAME user is first cleared, so the
            ``idx_user_models_one_active`` partial-unique invariant always holds (one active
model
            per user). Returns the row id, or None on failure.
            """
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if is_active:
                        # Demote the current active row first (NULL user_id = local: IS NULL
match).
                        if user_id is None:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
                                (version,))
                        else:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id = ? AND is_active = 1 AND version != ?",
                                (user_id, version))
                    cur.execute(
                        """INSERT INTO user_models
                           (user_id, version, baseline_version, feature_schema_hash,
                            fusion_config, classifier_json, promotion_evidence,
                            n_videos_at_fit, is_active)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                           ON CONFLICT(user_id, version) DO UPDATE SET
                             baseline_version=excluded.baseline_version,
                             feature_schema_hash=excluded.feature_schema_hash,
                             fusion_config=excluded.fusion_config,
                             classifier_json=excluded.classifier_json,
                             promotion_evidence=excluded.promotion_evidence,
                             n_videos_at_fit=excluded.n_videos_at_fit,
                             is_active=excluded.is_active""",
                        (user_id, version, baseline_version, feature_schema_hash,
                         json.dumps(fusion_config) if fusion_config is not None else None,
                         json.dumps(classifier_json) if classifier_json is not None else
None,
                         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
                         n_videos_at_fit, 1 if is_active else 0))
                    conn.commit()
                    rid = cur.execute(
                        "SELECT id FROM user_models WHERE "
                        + ("user_id IS NULL" if user_id is None else "user_id = ?")
                        + " AND version = ?",
                        ((version,) if user_id is None else (user_id, version))).fetchone()
                    return int(rid[0]) if rid else None
            except Exception as e:
                logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
                return None

        def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
            """Return the single active per-user model row for ``user_id`` (NULL = local
```

```
install),
703             with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
704             or None if the user has no active model. Does not touch the served pipeline."""
705             with self._connect() as conn:
706                 if user_id is None:
707                     row = conn.cursor().execute(
708                         "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
709                 else:
710                     row = conn.cursor().execute(
711                         "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
712                         (user_id,)).fetchone()
713                 if not row:
714                     return None
715                 d = dict(row)
716                 for col in ("fusion_config", "classifier_json", "promotion_evidence"):
717                     d[col] = json.loads(d[col]) if d[col] else None
718                 return d
719
```

4. assistant (2026-06-18T01:33:19.243Z)

I have the full database.py. Now let me examine the migration ladder carefully and find the callers. Let me look at how migrations interact with the existing-table assumptions and check the callers of these DB methods.

5. user (2026-06-18T01:33:21.274Z)

```
Found 19 files
.claude\worktrees\hunt-base\tests\test_reingest_safety.py
.claude\worktrees\hunt-base\tests\test_personalization_serving.py
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py
.claude\worktrees\hunt-base\tests\test_job_self_scheduling.py
.claude\worktrees\hunt-base\tests\test_database.py
.claude\worktrees\hunt-base\tests\test_async_jobs.py
.claude\worktrees\hunt-base\docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
.claude\worktrees\hunt-base\docs\UI_PROPOSAL.md
.claude\worktrees\hunt-base\docs\UI_ALPHA_EXECUTION_PLAN.md
.claude\worktrees\hunt-base\docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
.claude\worktrees\hunt-base\docs\PLATFORM_ARCHITECTURE.md
.claude\worktrees\hunt-base\docs\PERSONALIZATION_PLAN.md
.claude\worktrees\hunt-base\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
.claude\worktrees\hunt-base\docs\EXECUTION_POLICY.md
.claude\worktrees\hunt-base\docs\CODE_MAP.md
.claude\worktrees\hunt-base\docs\ANALYZERS_AND_RECONCILERS.md
.claude\worktrees\hunt-base\backend\main.py
.claude\worktrees\hunt-base\backend\infrastructure\database.py
.claude\worktrees\hunt-base\CLAUDE.md
```

6. user (2026-06-18T01:33:22.434Z)

```
.claude\worktrees\hunt-base\CLAUDE.md:42:  user_version` migration ladder in
`backend/infrastructure/database.py`.
.claude\worktrees\hunt-base\docs\ANALYZERS_AND_RECONCILERS.md:328:  `PRAGMA user_version`
migration ladder (`database.py:88`, `if version < 3:`) so applied corrections are
.claude\worktrees\hunt-base\docs\ANALYZERS_AND_RECONCILERS.md:522:  (`user_version < 3`); apply
behind `reconcile.apply`; **rollback = flip the flag.**
.claude\worktrees\hunt-base\backend\infrastructure\database.py:88:            # Versioned
migrations live here. PRAGMA user_version is the schema
.claude\worktrees\hunt-base\backend\infrastructure\database.py:91:            version =
cursor.execute("PRAGMA user_version").fetchone()[0]
.claude\worktrees\hunt-base\backend\infrastructure\database.py:121:
cursor.execute("PRAGMA user_version = 1")
```

.claude\worktrees\hunt-base\backend\infrastructure\database.py:148:
cursor.execute("PRAGMA user_version = 2")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:156:
cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:157:
cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:159:
cursor.execute("PRAGMA user_version = 3")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:168:
cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:169:
cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:178:
cursor.execute("PRAGMA user_version = 4")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:216:   CREATE
UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
.claude\worktrees\hunt-base\backend\infrastructure\database.py:226:
cursor.execute("PRAGMA user_version = 5")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:227:   # future: if
version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version = 6
.claude\worktrees\hunt-base\docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:94:>
`jobs` table at `user_version` 2, ThreadPoolExecutor(1), startup sweep for orphaned jobs, static
UI
.claude\worktrees\hunt-base\docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:108:
(`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
.claude\worktrees\hunt-base\docs\UI_PROPOSAL.md:54:| B1 | **Async job model ?
DEFERRED_HARDENING_PLAN #16** | The committed next PLATFORM slice; `POST /api/process` ? 202 +
`job_id`, `jobs` table (user_version 1?2), ThreadPoolExecutor, `GET /api/jobs/{id}`. **Approving
this proposal = greenlighting #16.** A 5?60 min blocking HTTP call cannot sit behind a tunnel
UI. |
.claude\worktrees\hunt-base\docs\UI_ALPHA_EXECUTION_PLAN.md:30:- [x] **PR-1 · B1 async job model
(#16):** `jobs` table (user_version 1?2), `POST /api/process` ? 202 + `job_id`, `GET
/api/jobs/{job_id}`, ThreadPoolExecutor worker, restart/crash state handling ? **MERGED
2026-06-11, PR #87** (CI green: 333 passed; machine-side real-video checks fold into L3)
.claude\worktrees\hunt-base\docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:55:
Canonical JSONL per rally (video_id MD5, rally_ordinal, sport, start/end, shot_count,
ending_reason/fault_tags, label_type, source, consent/training_opt_in, split=BY MATCH,
trajectory_ref, ...). Wire to durable per-video **event store** (the conversational-QA seam:
"extract once, query many"). DB delta: extend `play_segments`, new `highlights`/events table +
`PRAGMA user_version` migration. Reserve fault/event taxonomy now (service_fault etc.). No
Gemini-sourced rows in training views.
.claude\worktrees\hunt-base\docs\PERSONALIZATION_PLAN.md:83:**Per-user data model** (migrations
follow the `PRAGMA user_version` ladder; current = v5):
.claude\worktrees\hunt-base\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:65: even if unpopulated. ?
This is a real DB delta (a `PRAGMA user_version` migration): `play_segments` today has
.claude\worktrees\hunt-base\docs\PLATFORM_ARCHITECTURE.md:64:Grounding:
`backend/infrastructure/database.py` (schema + `PRAGMA user_version`
.claude\worktrees\hunt-base\docs\PLATFORM_ARCHITECTURE.md:221:graduate the `PRAGMA user_version`
migration discipline to **Alembic** under Postgres.
.claude\worktrees\hunt-base\docs\PLATFORM_ARCHITECTURE.md:475:| **P0** | ? **DONE ? Async job
model, single-tenant** (DEFERRED #16) | `jobs` table via `user_version` ladder; single-worker
`ThreadPoolExecutor`; `/api/process` ? 202 + job_id; `GET /api/jobs/{id}` status |
.claude\worktrees\hunt-base\tests\test_database.py:84:   version = conn.execute("PRAGMA
user_version").fetchone()[0]
.claude\worktrees\hunt-base\tests\test_database.py:106:   version = conn.execute("PRAGMA
user_version").fetchone()[0]
.claude\worktrees\hunt-base\tests\test_database.py:224:   conn.execute("ALTER TABLE videos DROP
COLUMN user_id")
.claude\worktrees\hunt-base\tests\test_database.py:225:   conn.execute("ALTER TABLE
candidate_segments DROP COLUMN user_id")
.claude\worktrees\hunt-base\tests\test_database.py:226:   conn.execute("PRAGMA user_version =
3")
.claude\worktrees\hunt-base\tests\test_database.py:234:   assert conn.execute("PRAGMA
user_version").fetchone()[0] == 5
.claude\worktrees\hunt-base\tests\test_database.py:294:def

```
test_upsert_is_idempotent_on_user_version(tmp_path):
.claude\worktrees\hunt-base\docs\EXECUTION_POLICY.md:89:  (user_version 3), `claim_due_job`
(atomic, earliest-due, compare-and-set), `set_job_waiting`,
.claude\worktrees\hunt-base\docs\CODE_MAP.md:216:[Omitted long matching line]
.claude\worktrees\hunt-base\docs\CODE_MAP.md:335:| Add a DB column/table |
`@backend/infrastructure/database.py::Database._init_db` migration ladder + bump `PRAGMA
user_version` (else version-assert tests fail) |
.claude\worktrees\hunt-base\docs\CODE_MAP.md:374:| `test_database.py` |
`infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
.claude\worktrees\hunt-base\tests\test_async_jobs.py:10:- migration: fresh DB lands at
user_version 2 with a jobs table; v1 DBs upgrade in place
.claude\worktrees\hunt-base\tests\test_async_jobs.py:83:        assert conn.execute("PRAGMA
user_version").fetchone()[0] == 5
.claude\worktrees\hunt-base\tests\test_async_jobs.py:101:    conn.execute("PRAGMA user_version =
1")
.claude\worktrees\hunt-base\tests\test_async_jobs.py:107:        assert c.execute("PRAGMA
user_version").fetchone()[0] == 5
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py:275:        assert
conn.execute("PRAGMA user_version").fetchone()[0] == 5
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py:279:    # A pre-migration legacy DB
(only `videos`, user_version 0) must run the FULL ordered
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py:287:    conn.execute("PRAGMA
user_version = 0")
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py:292:        assert c.execute("PRAGMA
user_version").fetchone()[0] == 5
```

7. assistant (2026-06-18T01:33:27.150Z)

Let me carefully analyze the migration ladder for the partial-migration-on-error and ordering concerns. A key thing I want to verify: the `if version < N` blocks all read the SAME `version` captured once at line 91, and each block is keyed off that single read. Let me think about the v5 migration's interaction with the `_connect` context manager's transaction scope, and the claim_due_job race window. Let me look at the worker loop and the test for migrations to understand the contracts.

8. user (2026-06-18T01:33:28.203Z)

```
1       import sqlite3
2       from backend.infrastructure.database import Database
3
4       def test_database_init(tmp_path):
5           db_path = tmp_path / "test.db"
6           Database(str(db_path))  # instantiation creates the schema
7
8           conn = sqlite3.connect(str(db_path))
9           cursor = conn.cursor()
10          cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
11          tables = [r[0] for r in cursor.fetchall()]
12          assert "videos" in tables
13          assert "play_segments" in tables
14          conn.close()
15
16      def test_database_video(tmp_path):
17          db = Database(str(tmp_path / "test.db"))
18
19          vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
20          assert vid_added is True
21
22          video = db.get_video("vid1")
23          assert video["id"] == "vid1"
24          assert video["filepath"] == "vid1.mp4"
25          assert video["status"] == "pending"
26          assert video["validation_results"] == [{"val": 1}]
27
```

```
        db.clear_video_data("vid1")
        assert db.get_video("vid1") is None

    def test_database_play_segment(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        db.add_video("vid1", "vid1.mp4", "pending", [])

        sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
        assert sid is not None

        db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})

        segments = db.query_play_segments("vid1", {"server": "PlayerA"})
        assert len(segments) == 1
        assert segments[0]["id"] == sid
        assert segments[0]["start_time"] == 10.0
        assert segments[0]["server"] == "PlayerA"
        assert segments[0]["metadata"]["foo"] == "bar"

        assert len(segments[0]["events"]) == 1
        assert segments[0]["events"][0]["type"] == "smash"
        assert segments[0]["events"][0]["details"]["speed"] == 300


    # --- candidate_segments schema + behavior ----------------------------------

    # The schema contract: any future change to candidate_segments columns must
    # consciously update this set, otherwise the test below fails (regression guard).
    EXPECTED_CANDIDATE_COLUMNS = {
        "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
        "duration", "confidence", "stats", "detection_params",
        "confirmed_segment_id", "human_label", "notes", "created_at",
        "user_id",  # v4: personalization groundwork (nullable; NULL = local install)
    }


    def test_candidate_schema_contract(tmp_path):
        Database(str(tmp_path / "test.db"))  # instantiation creates the schema
        conn = sqlite3.connect(str(tmp_path / "test.db"))
        cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
        conn.close()

        names = {c[1] for c in cols}
        assert names == EXPECTED_CANDIDATE_COLUMNS

        # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
pk)
        by_name = {c[1]: c for c in cols}
        assert by_name["id"][5] == 1  # primary key
        for col in ("video_id", "detector", "start_time", "end_time", "duration"):
            assert by_name[col][3] == 1, f"{col} should be NOT NULL"


    def test_schema_version_is_current(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))
        conn = sqlite3.connect(str(db_path))
        version = conn.execute("PRAGMA user_version").fetchone()[0]
        conn.close()
        # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
        assert version == 5


    def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
        db_path = tmp_path / "legacy.db"
```

```
92          # Simulate a pre-v1 database that only has the old tables and no version.
93          conn = sqlite3.connect(str(db_path))
94          conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
95          conn.commit()
96          conn.close()
97
98          # Opening with the new Database should add candidate_segments without error...
99          Database(str(db_path))
100         # ...and running init again must be a no-op (idempotent).
101         Database(str(db_path))
102
103         conn = sqlite3.connect(str(db_path))
104         tables = [r[0] for r in conn.execute(
105             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
106         version = conn.execute("PRAGMA user_version").fetchone()[0]
107         conn.close()
108         assert "candidate_segments" in tables
109         assert "jobs" in tables  # v2 (async job model, #16)
110         assert "user_models" in tables  # v5 (per-user model store)
111         assert version == 5      # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
112
113
114     def test_candidate_round_trip(tmp_path):
115         db = Database(str(tmp_path / "test.db"))
116         db.add_video("vid1", "vid1.mp4", "processed", [])
117
118         cid = db.add_candidate_segment(
119             "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
120             stats={"peak_velocity": 0.8, "track_id_count": 2},
121             detection_params={"frame_skip": 3},
122         )
123         assert cid is not None
124
125         cands = db.query_candidate_segments("vid1")
126         assert len(cands) == 1
127         c = cands[0]
128         assert c["detector"] == "yolo_hybrid"
129         assert c["start_time"] == 10.0
130         assert c["duration"] == 15.0
131         assert c["confidence"] == 0.42
132         assert c["stats"]["peak_velocity"] == 0.8
133         assert c["detection_params"]["frame_skip"] == 3
134         assert c["confirmed_segment_id"] is None
135
136         # detector filter
137         assert db.query_candidate_segments("vid1", detector="motion") == []
138         assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
139
140
141     def test_candidate_link_and_unlabeled_filter(tmp_path):
142         db = Database(str(tmp_path / "test.db"))
143         db.add_video("vid1", "vid1.mp4", "processed", [])
144
145         cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
146         sid = db.add_play_segment("vid1", 1.0, 4.0)
147
148         assert db.link_candidate_to_segment(cid, sid) is True
149         assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
150
151         # only_unlabeled returns it while human_label is NULL
152         assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
153         with db._get_connection() as conn:
154             conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
155                          ("true_rally", cid))
```

```
156             conn.commit()
157         assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
158
159
160     def test_candidate_cascade_on_video_delete(tmp_path):
161         db = Database(str(tmp_path / "test.db"))
162         db.add_video("vid1", "vid1.mp4", "processed", [])
163         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
164
165         db.clear_video_data("vid1")
166         assert db.query_candidate_segments("vid1") == []
167
168
169     # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
170
171     def _table_cols(db_path, table):
172         conn = sqlite3.connect(str(db_path))
173         cols = {c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()}
174         conn.close()
175         return cols
176
177
178     def test_v4_user_id_columns_present_and_nullable(tmp_path):
179         db_path = tmp_path / "test.db"
180         Database(str(db_path))
181         assert "user_id" in _table_cols(db_path, "videos")
182         assert "user_id" in _table_cols(db_path, "candidate_segments")
183
184         # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
185         conn = sqlite3.connect(str(db_path))
186         for table in ("videos", "candidate_segments"):
187             by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
188             assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
189         conn.close()
190
191
192     def test_v4_existing_writers_leave_user_id_null(tmp_path):
193         """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
194         row a current writer produces is local (NULL) ? byte-identical to pre-v4
behaviour."""
195         db_path = tmp_path / "test.db"
196         db = Database(str(db_path))
197         db.add_video("vid1", "vid1.mp4", "processed", [])
198         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
199
200         conn = sqlite3.connect(str(db_path))
201         vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1'").fetchone()[0]
202         cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1'").fetchone()[0]
203         conn.close()
204         assert vu is None
205         assert cu is None
206         # Existing readers are unaffected (no user_id filter in the query path).
207         assert len(db.query_candidate_segments("vid1")) == 1
208
209
210     def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
211         """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
212         columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
213         db_path = tmp_path / "legacy_v3.db"
214         db = Database(str(db_path))  # fresh DB is born at the current version; simulate a
```

```
v3 state
        # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
        db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
        db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
        conn = sqlite3.connect(str(db_path))
        conn.execute("DROP INDEX IF EXISTS idx_videos_user")
        conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
        conn.execute("DROP TABLE IF EXISTS user_models")
        # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
        # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
        conn.execute("ALTER TABLE videos DROP COLUMN user_id")
        conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
        conn.execute("PRAGMA user_version = 3")
        conn.commit()
        conn.close()
        assert "user_id" not in _table_cols(db_path, "videos")

        # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
        db2 = Database(str(db_path))
        conn = sqlite3.connect(str(db_path))
        assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
        assert "user_models" in {r[0] for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
        conn.close()
        assert "user_id" in _table_cols(db_path, "videos")
        assert "user_id" in _table_cols(db_path, "candidate_segments")

        v = db2.get_video("vidA")
        assert v is not None and v["validation_results"] == [{"v": 1}]
        cands = db2.query_candidate_segments("vidA")
        assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3


    # --- v5: per-user model store CRUD ------------------------------------------------

    def test_user_models_schema_contract(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))
        cols = _table_cols(db_path, "user_models")
        assert cols == {
            "id", "user_id", "version", "baseline_version", "feature_schema_hash",
            "fusion_config", "classifier_json", "promotion_evidence",
            "n_videos_at_fit", "is_active", "created_at",
        }


    def test_upsert_and_get_active_user_model_round_trip(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        rid = db.upsert_user_model(
            "user-1", version=1, baseline_version="base-2026-06",
            feature_schema_hash="abc123",
            fusion_config={"min_crossings": 3, "cue_flags": {"person": True}},
            classifier_json={"model": "logreg", "w": [0.1, 0.2], "b": 0.0},
            promotion_evidence={"promote": True, "n": 6},
            n_videos_at_fit=6, is_active=True,
        )
        assert rid is not None

        m = db.get_active_user_model("user-1")
        assert m is not None
        assert m["user_id"] == "user-1"
        assert m["version"] == 1
        assert m["baseline_version"] == "base-2026-06"
```

```
        assert m["feature_schema_hash"] == "abc123"
        assert m["fusion_config"] == {"min_crossings": 3, "cue_flags": {"person": True}}
        assert m["classifier_json"]["w"] == [0.1, 0.2]
        assert m["promotion_evidence"]["promote"] is True
        assert m["n_videos_at_fit"] == 6
        assert m["is_active"] == 1


    def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        assert db.get_active_user_model("nobody") is None

        # An inactive model is not returned as active.
        db.upsert_user_model("user-2", version=1, is_active=False)
        assert db.get_active_user_model("user-2") is None


    def test_upsert_is_idempotent_on_user_version(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        rid1 = db.upsert_user_model("user-3", version=1, n_videos_at_fit=4, is_active=True)
        rid2 = db.upsert_user_model(
            "user-3", version=1, n_videos_at_fit=7, is_active=True,
            fusion_config={"min_crossings": 5},
        )
        # Same (user, version) ? same row updated in place, not a duplicate.
        assert rid1 == rid2
        m = db.get_active_user_model("user-3")
        assert m["n_videos_at_fit"] == 7
        assert m["fusion_config"] == {"min_crossings": 5}

        conn = sqlite3.connect(str(tmp_path / "test.db"))
        count = conn.execute(
            "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-3'").fetchone()[0]
        conn.close()
        assert count == 1


    def test_activating_new_version_demotes_prior_active(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        db.upsert_user_model("user-4", version=1, n_videos_at_fit=5, is_active=True)
        db.upsert_user_model("user-4", version=2, n_videos_at_fit=9, is_active=True)

        active = db.get_active_user_model("user-4")
        assert active["version"] == 2  # the newer model is the single active one

        # Exactly one active row for the user (the partial-unique invariant holds).
        conn = sqlite3.connect(str(tmp_path / "test.db"))
        n_active = conn.execute(
            "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-4' AND is_active =
1").fetchone()[0]
        conn.close()
        assert n_active == 1


    def test_null_user_id_local_install_round_trip(tmp_path):
        """NULL user_id = the local install. upsert/get must treat IS NULL correctly (not
'=NULL')."""
        db = Database(str(tmp_path / "test.db"))
        db.upsert_user_model(None, version=1, fusion_config={"min_crossings": 2},
                             n_videos_at_fit=5, is_active=True)
        m = db.get_active_user_model(None)
        assert m is not None
        assert m["user_id"] is None
        assert m["fusion_config"] == {"min_crossings": 2}
        # A named user does NOT see the local row.
```

```
340         assert db.get_active_user_model("user-x") is None
341
342
343     def test_user_model_json_columns_default_none(tmp_path):
344         """A config-only model (no classifier) stores NULL classifier_json and deserialises
to None."""
345         db = Database(str(tmp_path / "test.db"))
346         db.upsert_user_model("user-5", version=1, fusion_config={"min_crossings": 4},
347                              n_videos_at_fit=5, is_active=True)
348         m = db.get_active_user_model("user-5")
349         assert m["classifier_json"] is None
350         assert m["promotion_evidence"] is None
351         assert m["fusion_config"] == {"min_crossings": 4}
352
```

9. assistant (2026-06-18T01:33:32.228Z)

Now I understand the test contracts well. Let me focus carefully on the key bug-class concerns. Let me examine the `claim_due_job` and `_connect` transaction scope interactions, and the `upsert_user_model` demotion + active-row invariant under one specific scenario. Let me look at the worker loop that calls these.

10. user (2026-06-18T01:33:33.525Z)

```
1       """Tests for the v3 self-scheduling job substrate (EXECUTION_POLICY.md P2).
2
3       Covers the policy column round-trip, set_job_waiting, and the atomic claim_due_job
4       (queued + due-waiting claimable, not-yet-due skipped, drained-then-None, compare-and-
set).
5       """
6       from backend.infrastructure.database import Database
7
8       POLICY = {"mode": "wait_and_resume", "op_timeout_sec": 90.0, "max_retries": 3}
9
10
11      def _db(tmp_path):
12          return Database(str(tmp_path / "jobs.db"))
13
14
15      def test_policy_roundtrips_through_create_and_get(tmp_path):
16          db = _db(tmp_path)
17          assert db.create_job("j1", "/v.mp4", policy=POLICY)
18          assert db.get_job("j1")["policy"] == POLICY
19
20
21      def test_create_without_policy_is_none(tmp_path):
22          db = _db(tmp_path)
23          db.create_job("j0", "/v.mp4")
24          assert db.get_job("j0")["policy"] is None
25
26
27      def test_set_job_policy_replaces(tmp_path):
28          db = _db(tmp_path)
29          db.create_job("j1", "/v.mp4")
30          assert db.set_job_policy("j1", {"mode": "abort"})
31          assert db.get_job("j1")["policy"] == {"mode": "abort"}
32
33
34      def test_set_job_waiting_sets_status_and_due(tmp_path):
35          db = _db(tmp_path)
36          db.create_job("j1", "/v.mp4")
37          assert db.set_job_waiting("j1", delay_sec=3600, progress="waiting on gemini")
38          row = db.get_job("j1")
39          assert row["status"] == "waiting" and row["next_poll_at"] is not None
40          assert row["progress"] == "waiting on gemini"
```

```
41
42
43      def test_claim_due_job_none_when_empty(tmp_path):
44          assert _db(tmp_path).claim_due_job() is None
45
46
47      def test_claim_due_job_claims_queued_and_flips_running(tmp_path):
48          db = _db(tmp_path)
49          db.create_job("j1", "/v.mp4", policy=POLICY)
50          claimed = db.claim_due_job()
51          assert claimed["id"] == "j1" and claimed["status"] == "running"
52          assert claimed["policy"] == POLICY                      # policy travels with the
claim
53          assert db.get_job("j1")["status"] == "running"
54
55
56      def test_claim_drains_then_returns_none(tmp_path):
57          db = _db(tmp_path)
58          db.create_job("a", "/a.mp4")
59          db.create_job("b", "/b.mp4")
60          got = {db.claim_due_job()["id"], db.claim_due_job()["id"]}
61          assert got == {"a", "b"}                                # both drained (order not
asserted)
62          assert db.claim_due_job() is None                       # nothing runnable left
63
64
65      def test_waiting_job_due_is_claimable(tmp_path):
66          db = _db(tmp_path)
67          db.create_job("j1", "/v.mp4")
68          db.set_job_waiting("j1", delay_sec=0)                   # due immediately
69          claimed = db.claim_due_job()
70          assert claimed is not None and claimed["id"] == "j1" and claimed["status"] ==
"running"
71
72
73      def test_waiting_job_not_due_is_skipped(tmp_path):
74          db = _db(tmp_path)
75          db.create_job("j1", "/v.mp4")
76          db.set_job_waiting("j1", delay_sec=3600)                # due in an hour
77          assert db.claim_due_job() is None                       # not yet due -> not
claimed
78          assert db.get_job("j1")["status"] == "waiting"          # left parked
79
80
81      def test_running_job_is_not_reclaimed(tmp_path):
82          db = _db(tmp_path)
83          db.create_job("j1", "/v.mp4")
84          db.claim_due_job()                                      # -> running
85          assert db.claim_due_job() is None                       # a running job is not
runnable
86
```

11. assistant (2026-06-18T01:33:43.514Z)

Now let me think carefully about a real concurrency/correctness concern in `claim_due_job`. The compare-and-set logic relies on `cur.rowcount` after the UPDATE. But there's a subtler issue I want to verify regarding the `next_poll_at <= CURRENT_TIMESTAMP` comparison vs the `datetime('now', ...)` format used in `set_job_waiting`. Let me verify the timestamp formats are comparable.

In `set_job_waiting`, `next_poll_at = datetime('now', '+N seconds')` produces `'YYYY-MM-DD HH:MM:SS'`. In `claim_due_job`, the comparison is `next_poll_at <= CURRENT_TIMESTAMP`. `CURRENT_TIMESTAMP` in SQLite is also `'YYYY-MM-DD HH:MM:SS'` UTC. So those are comparable as strings ? good.

Now let me look at a real bug-class concern in the migration ladder: what happens when a migration's `if version < N` block partially fails. Let me check the `created_at` ordering issue in `claim_due_job` and the worker loop. Let me read the worker loop file.

12. user (2026-06-18T01:33:43.856Z)

No files found

13. user (2026-06-18T01:33:43.931Z)

```
.claude\worktrees\hunt-base\backend\main.py:241:    NOT a failure. The worker persists
`next_poll_at` + `progress` (via `set_job_waiting`),
.claude\worktrees\hunt-base\backend\main.py:242:    releases the thread, and re-claims the job
when due (`claim_due_job`). No work to date
.claude\worktrees\hunt-base\backend\main.py:258:    re-claimed by `claim_due_job` (queued OR
resumed-from-waiting) runs identically to
.claude\worktrees\hunt-base\backend\main.py:269:    """Run ONE job already claimed
(`status='running'`) by `claim_due_job`, recording its
.claude\worktrees\hunt-base\backend\main.py:284:        db_instance.mark_job_done(job_id,
result)
.claude\worktrees\hunt-base\backend\main.py:288:        db_instance.set_job_waiting(job_id,
w.delay_sec, progress=w.progress)
.claude\worktrees\hunt-base\backend\main.py:291:        db_instance.mark_job_failed(job_id,
f"{type(e).__name__}: {e}")
.claude\worktrees\hunt-base\backend\main.py:305:        job = db_instance.claim_due_job()
.claude\worktrees\hunt-base\backend\main.py:322:        delay =
db_instance.seconds_until_next_due()
.claude\worktrees\hunt-base\backend\main.py:353:    worker drains it via `claim_due_job`
(EXECUTION_POLICY.md P3 self-scheduling loop).
.claude\worktrees\hunt-base\backend\main.py:786:    swept = db_instance.fail_stale_jobs()
.claude\worktrees\hunt-base\backend\infrastructure\database.py:154:                # re-claims
it via `claim_due_job` when due ? with NO master node: the table IS
.claude\worktrees\hunt-base\backend\infrastructure\database.py:503:    def mark_job_done(self,
job_id: str, result: Dict[str, Any]) -> bool:
.claude\worktrees\hunt-base\backend\infrastructure\database.py:518:    def mark_job_failed(self,
job_id: str, error: str) -> bool:
.claude\worktrees\hunt-base\backend\infrastructure\database.py:557:    def set_job_waiting(self,
job_id: str, delay_sec: float, progress: Optional[str] = None) -> bool:
.claude\worktrees\hunt-base\backend\infrastructure\database.py:559:        worker.
`claim_due_job` re-picks it when due ? cooperative, no master. ``next_poll_at`` is
.claude\worktrees\hunt-base\backend\infrastructure\database.py:574:    def
seconds_until_next_due(self) -> Optional[float]:
.claude\worktrees\hunt-base\backend\infrastructure\database.py:591:    def claim_due_job(self)
-> Optional[Dict[str, Any]]:
.claude\worktrees\hunt-base\backend\infrastructure\database.py:616:    def fail_stale_jobs(self,
reason: str = "interrupted by server restart") -> int:
```

14. assistant (2026-06-18T01:33:46.444Z)

Let me read the worker loop in main.py around the relevant lines, particularly the `fail_stale_jobs` interaction with `waiting` jobs and the `claim_due_job` race window.

15. user (2026-06-18T01:33:47.899Z)

```
230                 val_results=val_results, val_context=val_context,
231                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
232                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
233             )
234             if paths and isinstance(response, dict):
235                 response["reports"] = paths
236
237
238     class JobWaiting(Exception):
239         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
```

```
240         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
241         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
242         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
243         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
244
245         The wiring is additive: the production segmenter path never raises this today (zero
246         behaviour change), so a job runs exactly as before. The hook is what a later slice
247         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
248         """
249
250         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
251             super().__init__(f"job parked for {delay_sec:.0f}s")
252             self.delay_sec = max(0.0, float(delay_sec))
253             self.progress = progress
254
255
256     def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
257         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
258         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
259         the original submit. The row stores exactly the ProcessRequest fields."""
260         return ProcessRequest(
261             video_path=job["video_path"],
262             player_pool=job.get("player_pool"),
263             max_frames=job.get("max_frames"),
264             segmenter=job.get("segmenter"),
265         )
266
267
268     def _run_claimed_job(job: Dict[str, Any]) -> None:
269         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
270         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
271
272         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
273         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
274         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
275         job self-scheduled and will be re-claimed when `next_poll_at` is due.
276         """
277         job_id = job["id"]
278         req = _job_to_request(job)
279         try:
280             result = _execute_processing(
281                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
282             if result.get("video_id"):
283                 db_instance.set_job_video_id(job_id, result["video_id"])
284             db_instance.mark_job_done(job_id, result)
285         except JobWaiting as w:
286             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289         except Exception as e:
290             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294     def _drain_due_jobs() -> int:
295         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
296         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
297
```

```
        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
        up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the
clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        ran = 0
        while True:
            job = db_instance.claim_due_job()
            if job is None:
                break
            ran += 1
            _run_claimed_job(job)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _schedule_next_drain_if_waiting()
        return ran


    def _schedule_next_drain_if_waiting() -> None:
        """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
        self-scheduled job resumes with no further client submit. The drain itself re-checks
        due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
        try:
            delay = db_instance.seconds_until_next_due()
        except Exception as e:  # pragma: no cover - defensive; never wedge the worker
            logger.error(f"Could not compute next-due delay: {e}")
            return
        if delay is None:
            return  # nothing waiting
        # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
        _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))


    def _arm_wake_timer(delay_sec: float) -> None:
        """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
        scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
        monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
        timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
        timer.daemon = True
        timer.start()


    #: A parked job further out than this is re-checked by a capped wake rather than one
long
    #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
    _MAX_DRAIN_WAKE_SEC = 60.0


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
```

```
352         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
353         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
354         """
355         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
356         try:
357             validate_input_path(req.video_path, allowed_root=allowed_root)
358         except ValueError as e:
359             raise HTTPException(status_code=400, detail=str(e)) from e
```

16. assistant (2026-06-18T01:33:58.213Z)

Now I have a clear picture. Let me carefully examine the `fail_stale_jobs` crash-recovery logic for a real bug. It sweeps `status IN ('queued', 'running')` to 'failed'. But notice: a job in ``'waiting'`` status is NOT swept. That's by design (waiting jobs are persistent across restarts).

Now let me focus on the most concrete migration-ladder concern. Let me re-examine the v3 ALTER block. SQLite executes DDL like `ALTER TABLE` and `CREATE TABLE` ? in SQLite, DDL statements are transactional. But there's a subtle issue with `ALTER TABLE ... ADD COLUMN` inside the migration: if the `jobs` table from v2 already has the `policy` column (e.g., a partially-applied v3 migration that crashed AFTER the ALTER but BEFORE the PRAGMA bump), re-running v3 would fail with "duplicate column name". Let me verify this partial-migration scenario is reachable.

The whole `_init_db` runs inside `with self._connect()` ? `with conn:` which is a single transaction. In SQLite, an exception inside the `with conn:` block triggers a rollback. So if the migration partially completes and then errors, the transaction rolls back ? meaning the PRAGMA bumps and DDL roll back together. Let me verify whether DDL + PRAGMA user_version are atomic.

Actually, there's a critical subtlety: **`PRAGMA user_version = N` is NOT transactional in the way data is.** Let me verify this carefully because it's central to the partial-migration bug class.

17. user (2026-06-18T01:34:02.565Z)

```
initial: 0
mid-txn user_version: 5
caught: boom
after rollback user_version: 5
table t exists: True
```

18. assistant (2026-06-18T01:34:08.791Z)

This is very interesting. In this Python sqlite3 driver behavior, `PRAGMA user_version` is NOT rolled back, and notably **the CREATE TABLE was also committed despite the rollback**. Wait ? that's because Python's sqlite3 module does NOT begin an implicit transaction for DDL statements in the default `isolation_level` mode prior to Python 3.12. Let me check the Python version and dig deeper, because this is the crux of partial-migration safety.

19. user (2026-06-18T01:34:15.191Z)

```
Python 3.13.13
sqlite_version: 3.50.4
default isolation_level: ''
caught: crash mid-migration
jobs cols after failed migration: ['id', 'status', 'policy']
user_version: 2
RE-RUN v3 FAILED: duplicate column name: policy
```

20. assistant (2026-06-18T01:34:21.159Z)

This is a real and important finding. On Python 3.13 (and 3.12+), the `ALTER TABLE` DDL was **committed despite the rollback** ? the `policy` column persists, but `user_version` stayed at

2. On re-open, the v3 block runs again and fails with "duplicate column name: policy", permanently bricking the DB at version 2.

But wait ? I need to be careful about Python version dependence. The behavior differs between Python <3.12 and ?3.12 due to the change in how `sqlite3` handles transactions for DDL. The project says "Python 3.8+". Let me verify the behavior on the older legacy `isolation_level` semantics, because in Python <3.12, DDL statements implicitly committed any pending transaction (so the ALTER would commit immediately regardless). Let me confirm this is a real defect across versions, not just a 3.13 artifact.

21. user (2026-06-18T01:34:31.953Z)

```
caught: interrupted before v5 completes
user_version after: 4
videos cols: ['id', 'filepath', 'status', 'user_id']
reopened version: 4
```

22. assistant (2026-06-18T01:34:44.161Z)

Interesting ? in this case `user_version` reached 4 because the `PRAGMA user_version = 4` was executed and DDL auto-committed. So when v4 completes (including its PRAGMA bump) before v5 fails, the version IS at 4, and the next reopen correctly skips v4 and only re-runs v5. That's safe for the *boundary between complete migration blocks*.

The dangerous case is failure *within a single migration block* ? between an `ALTER`/`CREATE` and that block's own `PRAGMA user_version = N`. Let me verify which migration blocks have multiple DDL statements before the version bump where re-running a partially-applied block fails. Let me re-examine each block:

- **v1**: `CREATE TABLE IF NOT EXISTS` + `CREATE INDEX IF NOT EXISTS` + PRAGMA. All idempotent (IF NOT EXISTS). Safe to re-run.
- **v2**: `CREATE TABLE jobs` (no IF NOT EXISTS? let me check)... it IS `CREATE TABLE IF NOT EXISTS jobs` + `CREATE INDEX IF NOT EXISTS` + PRAGMA. Safe.
- **v3**: `ALTER TABLE jobs ADD COLUMN policy` ? **NOT idempotent**. `ALTER ... ADD COLUMN next_poll_at` ? **NOT idempotent**. + index + PRAGMA. **Re-running a partially-applied v3 fails.**
- **v4**: `ALTER TABLE videos ADD COLUMN user_id` + `ALTER TABLE candidate_segments ADD COLUMN user_id` ? **NOT idempotent.** + indexes + PRAGMA. **Re-running a partially-applied v4 fails.**
- **v5**: all `CREATE ... IF NOT EXISTS`. Safe.

So the concrete bug is: **the migration ladder is NOT crash-safe at the v3 and v4 ALTER boundaries.** Let me confirm whether the `with conn:` transaction actually fails to protect this. The key insight from my test: under Python's sqlite3 (all versions in scope, since the project targets 3.8+), DDL statements within a `with conn:` block can be committed even when a later statement in the same block raises and triggers rollback ? because the version bump is the only thing that gates re-running.

Let me verify the precise Python <3.12 behavior is also broken, since the project supports 3.8+. Under <3.12, `isolation_level=''` means DML opens a transaction but **DDL statements implicitly commit** the pending transaction first. Let me confirm.

23. user (2026-06-18T01:34:53.787Z)

```
first run errored at stmt2: duplicate column name: user_id
version: 3
videos cols: ['id', 'user_id']
reopen version: 3
RE-OPEN BRICKED:  duplicate column name: user_id
```

24. assistant (2026-06-18T01:35:11.162Z)

The mechanism is confirmed. However, I need to be honest about the **trigger realism**. The realistic trigger is: a crash/interruption (process kill, disk-full, power loss, `OperationalError` like `database is locked` or `disk I/O error`) occurring *mid-block* between

an ALTER and that block's PRAGMA bump. When that happens, the applied-but-not-recorded ALTER permanently bricks the DB because the re-run hits "duplicate column name."

This is a genuine latent defect (the migration ladder is not idempotent at the v3/v4 ALTER boundaries), but I should assess severity honestly. It requires a crash at a narrow window during a one-time upgrade. It's a medium-severity robustness defect ? the kind a maintainer would accept as real (migration ladders are conventionally expected to be re-runnable/crash-safe). The fix is straightforward: guard ALTER columns with existence checks, or make the version bump happen atomically with the DDL.

Now let me look for any **more concrete, always-triggerable** bug. Let me re-examine `claim_due_job` line 602: `ORDER BY COALESCE(next_poll_at, created_at) ASC`. For a `queued` job, `next_poll_at` is NULL, so it orders by `created_at`. For a `waiting` job, it orders by `next_poll_at`. Comparing a `next_poll_at` (datetime string) with a `created_at` (also datetime string from CURRENT_TIMESTAMP) ? both `'YYYY-MM-DD HH:MM:SS'`, comparable. Fine.

Let me check the `seconds_until_next_due` and a subtle issue with `set_job_waiting`'s `COALESCE(?, progress)` ? if `progress` param is None, it keeps existing progress. Fine.

Now let me scrutinize a real schema/row contract concern: `get_job` deserializes `result`, `player_pool`, `policy` but NOT... let me check what `claim_due_job` returns ? it calls `get_job`. Fine. Let me check whether there's a contract drift between what `claim_due_job` reads and concurrent claim. Let me look at the `_get_connection` ? each call opens a new connection. In `claim_due_job`, the SELECT + UPDATE + commit happen on one connection inside `with conn:`. But the SELECT is a plain read (no `BEGIN IMMEDIATE`). Two workers could both SELECT the same row, then both UPDATE with compare-and-set. The compare-and-set `WHERE id=? AND status=?` protects against double-claim via `rowcount != 1`. Good design.

Let me now check the migration test that simulates a v3 DB to be sure the partial-failure path isn't already handled somewhere. Let me also double check there's no `IF NOT EXISTS` equivalent guard. Let me re-read lines 150-178 precisely.

25. user (2026-06-18T01:35:12.786Z)

```
150                 if version < 3:
151                     # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152                     # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153                     # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154                     # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155                     # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156                     cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157                     cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158                     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159                     cursor.execute("PRAGMA user_version = 3")
160
161                 if version < 4:
162                     # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163                     # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164                     # path can scope rows to an owner. **NULL = local install = byte-
identical to
165                     # today** ? every existing row stays NULL and every existing query
(which never
166                     # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167                     # Additive ONLY: no served-behavior change rides on this slice.
168                     cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169                     cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170                     # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
```

```
path free
171                     # of index churn while making the eventual per-user lookups cheap.
172                     cursor.execute(
173                         "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174                         "WHERE user_id IS NOT NULL")
175                     cursor.execute(
176                         "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177                         "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178                     cursor.execute("PRAGMA user_version = 4")
179
```

26. assistant (2026-06-18T01:35:25.347Z)

Confirmed. The v3 and v4 blocks contain non-idempotent `ALTER TABLE ADD COLUMN` statements before the version bump, with no existence guard. This is a real, if narrow-trigger, crash-safety defect.

Now let me look harder for a bug with a **deterministic, always-fires trigger** rather than a crash-window one. Let me reconsider the `upsert_user_model` active-row invariant. The demotion clears prior active rows with `version != ?`. Then the INSERT...ON CONFLICT sets `is_active=1`. Consider this scenario: a user has version=1 active. Caller calls `upsert_user_model(user, version=1, is_active=False)` to *deactivate*. The `is_active` is False, so the demotion block is skipped, and the UPSERT sets `is_active=0`. Fine.

Now consider: caller calls `upsert_user_model(user, version=2, is_active=True)` while v1 is active. Demotion clears v1 (`version != 2`). UPSERT inserts v2 active. Good ? one active.

Now the subtle one: the demotion runs BEFORE the insert, and the demotion + insert + commit are all in one `with conn:` transaction. That's atomic. Good.

But wait ? there's an ordering subtlety with the partial-unique index `idx_user_models_one_active`. Consider upserting the SAME version that's already active for a different... no, the index is on `user_id WHERE is_active=1`. Since the demotion excludes the current version and the insert sets that version active, at commit there's exactly one active row. Fine. Actually let me check one tricky path: what if you `upsert_user_model(user, version=1, is_active=True)` when version=2 is currently the active one. Demotion clears rows `WHERE user_id=? AND is_active=1 AND version != 1` ? clears v2. Then UPSERT on (user, v1) sets is_active=1. One active (v1). Correct.

The `upsert_user_model` logic looks sound. Let me verify there are no other callers of these DB methods that violate a contract ? particularly `mark_job_done`/`set_job_video_id` ordering and the `result` JSON serialization. Let me check `_execute_processing`'s result for non-JSON-serializable content. Let me grep for the result dict construction.

27. user (2026-06-18T01:35:26.471Z)

```
770
771         # Initialize Global Config and DB
772         global global_config, db_instance
773         global_config = load_app_config()  # returns typed AppConfig
774
775         # The CLI --output-dir overrides the configured output directory. It now lives
776         # on the typed model (OutputConfig.output_dir), so every consumer ? including
777         # /api/compile ? reads the same value instead of a write-only runtime hack.
778         global_config.output.output_dir = args.output_dir
779         os.makedirs(global_config.output.output_dir, exist_ok=True)
780
781         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782         db_instance = Database(db_path)
783
784         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785         # the executor is in-process. Mark them failed so pollers see a terminal state.
786         swept = db_instance.fail_stale_jobs()
787         if swept:
```

```
788             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789
790         # CLI processing mode (no web UI needed)
791         if args.video_path and not args.server:
792             print(f"[CLI] Processing video file: {args.video_path}")
793             if not os.path.exists(args.video_path):
794                 print(f"[FAIL] Video file not found: {args.video_path}")
795                 return
796
797             video_id = compute_video_id(args.video_path)
798             was_reingest = db_instance.get_video(video_id) is not None
799
800             # Validate (with-context variant surfaces ffprobe_meta for the report)
801             print("[CLI] Validating...")
802             val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
803             failures = [r for r in val_results if not r.passed]
804
805             # Save video status
806             status = "failed" if failures else "processed"
807             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
808
809             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
```

28. assistant (2026-06-18T01:35:43.649Z)

This is fine. Now let me consider the `fail_stale_jobs` interaction with `waiting` jobs more carefully ? it's actually a potential bug. A job in `waiting` status survives a restart (intentionally). But after a restart, who re-arms the wake timer for waiting jobs? `fail_stale_jobs` runs at startup but only sweeps queued/running. The `_schedule_next_drain_if_waiting` is only called at the end of `_drain_due_jobs`. So on a fresh restart, if there are `waiting` jobs but no new submit, nothing drains them. But that's a design/feature concern in main.py, not a database.py migration/transaction defect, and it's out of my assigned class. Let me stay focused.

Let me verify one more migration-ladder concern: the `test_init_db_idempotent_and_upgrades_old_db` test creates a legacy `videos` table WITHOUT the `status NOT NULL` and without `validation_results`/`processed_at` columns. The v0?current upgrade adds `candidate_segments`, `jobs`, etc., but does it add the missing columns to the legacy `videos` table? The legacy table is `CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status TEXT)` ? missing `processed_at` and `validation_results`. The `_init_db` uses `CREATE TABLE IF NOT EXISTS videos (...)` which is a no-op since `videos` exists. Then v4 does `ALTER TABLE videos ADD COLUMN user_id`. So a legacy `videos` table ends up missing `processed_at` and `validation_results`. Then `add_video` does `INSERT INTO videos (id, filepath, status, validation_results)` ? which would fail because `validation_results` column doesn't exist!

Wait ? but the test at line 90-111 only checks tables/version, not that `add_video` works on the upgraded legacy DB. Let me verify this is a real reachable scenario vs. just a synthetic test. The legacy table in the test is synthetic. In reality, was there ever a shipped `videos` schema without `validation_results`? Let me check git history of the schema. Actually, this is speculative ? the "pre-v1" legacy shape is only constructed in the test. I won't report speculative schema drift without evidence that such a DB exists in the wild. Let me set that aside.

Let me re-examine the most defensible finding and check whether the test suite already exercises a v3-partial scenario, and whether there's precedent showing the maintainer considers crash-safety in scope. Let me check the `_get_connection` WAL + the fact that `_init_db` runs DDL ? and importantly whether autocommit mode affects this. Let me confirm the actual production exposure by checking the Python version requirement and how `with conn:` behaves with the default isolation level for the ALTER.

29. user (2026-06-18T01:35:45.588Z)

```
.claude\worktrees\hunt-base\pyproject.toml:21:requires-python = ">=3.10"
.claude\worktrees\hunt-base\backend\config\models.py:150:    # +0.047, sign 8/0 p=0.008, |?end|
5.7?3.8s) ? see docs/RALLY_QUALITY_RESEARCH.md.
.claude\worktrees\hunt-base\scripts\doctor.py:73:        print("[WARNING] Python 3.8+ is highly
recommended!")
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py:258:        # Window is [0.033,
3.967]; GT is the rally span the synthetic motion fills.
```

30. assistant (2026-06-18T01:35:56.471Z)

`requires-python = ">=3.10"`. So Python 3.10?3.13 are in scope. The behavior I demonstrated (DDL auto-commit leaving columns applied while version isn't bumped) holds across these. Under 3.10/3.11 with legacy isolation_level, `ALTER TABLE` implicitly commits the active transaction before executing, so the column is committed immediately ? making the partial-failure even MORE likely to leave a half-applied state. Under 3.12+, same observable outcome as I showed. The defect is real across the supported range.

I now have a solid, honest finding. Let me reconsider severity once more. The trigger is: an interruption (process kill / power loss / `disk I/O error` / `database is locked` timeout exceeded raising OperationalError) striking *between* the first ALTER and the version bump within v3 or v4, during a one-time schema upgrade. After that, every subsequent open of that DB fails in `_init_db` with "duplicate column name" ? the app cannot start (the DB is permanently bricked, no recovery without manual intervention). High impact, low-probability trigger ? I'll rate it medium severity.

Let me make sure I'm not missing an even simpler always-on bug. Let me re-check `seconds_until_next_due` for the empty-table case and `claim_due_job`'s `created_at` being NULL ordering. `created_at` has `DEFAULT CURRENT_TIMESTAMP` so it's never NULL for created jobs. Fine.

Let me verify there are no other ALTER-based migrations hiding elsewhere (e.g., reconcile or analyzers referenced in docs mentioning `user_version < 3`).

31. user (2026-06-18T01:35:57.964Z)

```
.claude\worktrees\hunt-base\backend\infrastructure\database.py:12:        self._init_db()
.claude\worktrees\hunt-base\backend\infrastructure\database.py:35:        conn.execute("PRAGMA
foreign_keys = ON")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:37:        conn.execute("PRAGMA
busy_timeout = 30000")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:39:
conn.execute("PRAGMA journal_mode = WAL")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:44:    def _init_db(self):
.claude\worktrees\hunt-base\backend\infrastructure\database.py:88:            # Versioned
migrations live here. PRAGMA user_version is the schema
.claude\worktrees\hunt-base\backend\infrastructure\database.py:91:            version =
cursor.execute("PRAGMA user_version").fetchone()[0]
.claude\worktrees\hunt-base\backend\infrastructure\database.py:121:
cursor.execute("PRAGMA user_version = 1")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:148:
cursor.execute("PRAGMA user_version = 2")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:156:
cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:157:
cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:159:
cursor.execute("PRAGMA user_version = 3")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:168:
cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:169:
cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:178:
cursor.execute("PRAGMA user_version = 4")
```

.claude\worktrees\hunt-base\backend\infrastructure\database.py:216:   CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
.claude\worktrees\hunt-base\backend\infrastructure\database.py:226:
cursor.execute("PRAGMA user_version = 5")
.claude\worktrees\hunt-base\backend\infrastructure\database.py:227:   # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version = 6

32. assistant (2026-06-18T01:36:07.846Z)

All migration logic lives in `database.py`. The only non-idempotent statements are the four `ALTER TABLE ... ADD COLUMN` in the v3 and v4 blocks (lines 156?157, 168?169).

I've thoroughly analyzed the migration ladder and transaction handling. My assessment:

1. **The v3/v4 ALTER crash-safety issue** is a genuine but narrow-trigger defect (requires interruption in a sub-second window during a one-time upgrade). It is the one real finding worth reporting ? a maintainer would agree the ladder should be idempotent at the ALTER boundaries, which is a well-understood convention violated here.

2. Everything else (claim_due_job compare-and-set, timestamp format comparability, upsert active-row invariant, UPSERT vs REPLACE, JSON (de)serialization, foreign_keys/WAL pragmas, fail_stale_jobs scope) is correct.

Let me reconsider whether the v3/v4 finding meets the bar of "a CONCRETE trigger reaches it." The trigger is real (any uncaught error mid-block: power loss, `disk I/O error`, SIGKILL, or even a `busy_timeout` exhaustion raising `OperationalError` mid-transaction during the upgrade). The consequence is a permanently unopenable DB. This clears the bar for a medium-severity report. I'll report exactly this one finding.

33. user (2026-06-18T01:36:21.716Z)

Structured output provided successfully

34. assistant (2026-06-18T01:36:34.543Z)

I completed the adversarial bug hunt for the SQLite migration ladder & transactions class.

Summary

I cross-cut the migration ladder and transaction handling across `backend/infrastructure/database.py` and its callers (`backend/main.py` worker loop, the test contracts in `tests/test_database.py`, `test_job_self_scheduling.py`). I empirically reproduced behaviors against CPython's sqlite3 to validate (not just reason about) the one concern that held up.

One finding reported (medium severity, confidence 0.7):

The migration ladder is not crash-safe at the v3/v4 boundaries. The v3 block (`backend/infrastructure/database.py:156-157`) and v4 block (`:168-169`) use bare, non-idempotent `ALTER TABLE ... ADD COLUMN` statements before their `PRAGMA user_version = N` bumps (`:159`, `:178`). Because CPython's sqlite3 commits DDL independently of the surrounding `with conn:` scope (confirmed across the supported 3.10?3.13 range), an interruption between an ALTER and its version bump leaves the column applied while `user_version` stays at the prior value. The next `Database(db_path)` construction re-runs the block and raises `duplicate column name`, permanently bricking the DB ? and since `_init_db` runs in `__init__`, the app can never start against that file. The v1/v2/v5 blocks correctly use `CREATE ... IF NOT EXISTS`, so only v3/v4 are exposed. Fix: guard the ALTERs with a `PRAGMA table_info` column-existence check (or catch the duplicate-column error per ALTER).

Verified-clean (no finding): `claim_due_job` compare-and-set + `rowcount` double-claim guard; `next_poll_at` vs `CURRENT_TIMESTAMP` string-timestamp comparability; `set_job_waiting`'s `datetime('now', ...)` UTC format; `upsert_user_model` demote-then-insert active-row invariant under the partial-unique index; UPSERT-not-REPLACE rowid preservation; `fail_stale_jobs` status scoping; FK-CASCADE/WAL pragmas; JSON (de)serialization round-trips. The synthetic "pre-v1

legacy videos table missing validation_results" path in
`test_init_db_idempotent_and_upgrades_old_db` is test-only with no evidence of such a DB
shipping, so I did not report it.

The single relevant file is `C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/hunt-base/backend/infrastructure/database.py`.